INTRODUCTION TO COMPILERS

By :

Dr. Sherin Omran

Evaluation plan

For Students by low 2012:

- Midterm exam: 10
- Final exam: 70
- Section: 10
- Lecture: 10

For Students by low 2018:

- Midterm exam: 20
- Final exam: 40
- Section: 15
- Lecture: 25

CHAPTER1
INTRODUCTION

History of Compilers

How are programming languages implemented?

Two major strategies:

- Interpreters (old and much less studied)
- Compilers (newer and very well understood with mathematical foundations).
- Interpreters run programs “as is”.
- Compilers do extensive preprocessi.ng

History of High-Level Languages

- In 1954:
  - ✓IBM developed the 704 machine.
  - ✓(It used floating point arithmetic hardware).
  - ✓The IBM 704 was able to process about 40,000 instructions per second.
- Problem:
  - ✓Software costs exceeded the hardware costs!.



IBM 704 machine

The Solution

- Enter “**Speed coding**”
- An interpreter ran **10-20 times slower** than hand-written **assembly.**
- The idea to solve that problem is to translate the high-level code to assembly.
- Many thought this impossible.
- It had already failed in other projects.

The first compiler

- FORTRAN
  - ✓Stands for **F**ormula Translation.
- 1958
  - ✓50% of all software is in FORTRAN.
  - ✓Modern compilers preserve the outlines of FORTRAN I.

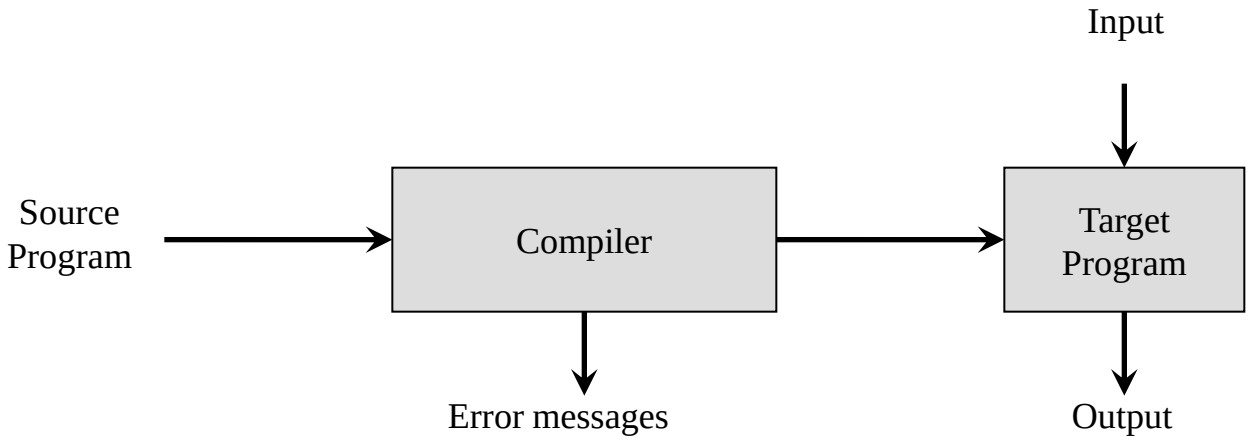
The Structure of a Compiler

1. Lexical analysis.
2. Syntax analysis (Parsing).
3. Semantic analysis.
4. Optimization.
5. Code generation.

The first 3, at least, can be understood by analogy to how humans comprehend English

What is a Compiler?

- A compiler is a program that can read a program in one language (the source language) and translate it into an equivalent program in another language (the target language).
- An important role of the compiler is to report any errors in the source program that it detects during the translation process.



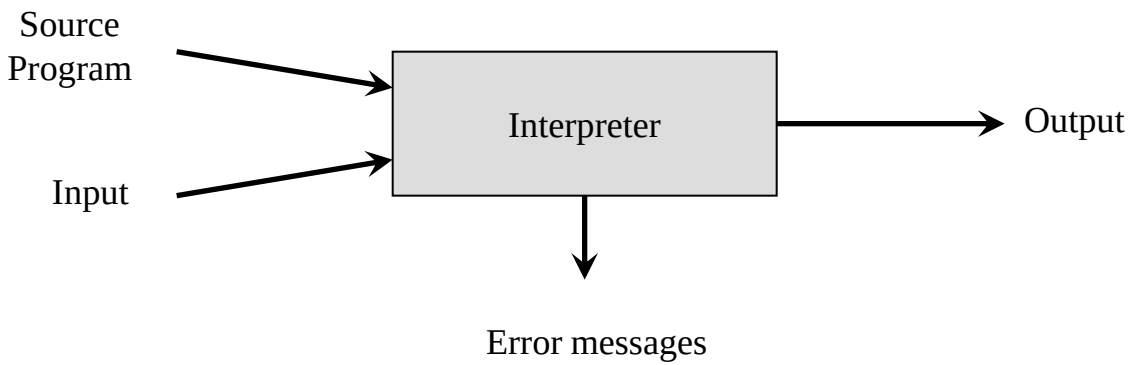
Why we use high-level language for programming?

Using a high-level language for programming has a large impact on how fast programs can be developed. The main reasons for this are:

1. Compared to machine language, the notation used by programming languages is closer to the way humans think about problems.
2. The compiler can spot some obvious programming mistakes.
3. Programs written in a high-level language tend to be shorter than equivalent programs written in machine language.
4. The same program can be compiled to many different machine languages and, hence, be brought to run on many different machines.

What is the Interpreter?

- An interpreter is another common kind of language processor.
- Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.



Compiler vs. Interpreter

Compiler

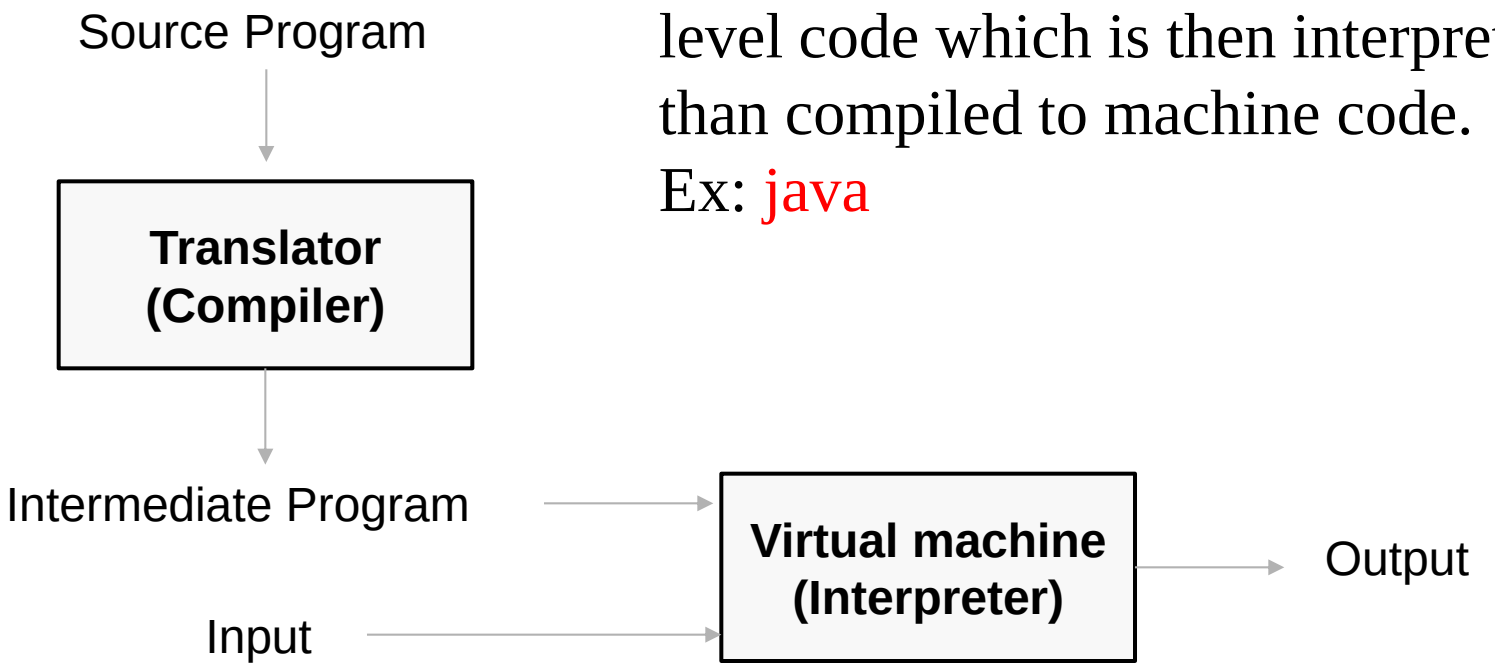
- **Takes Entire program as input**
- It is **Faster**
- intermediate object code is **generated.**
- Required **more** memory Due to intermediate object code
- Program **not need compile** every time
- **Errors** are displayed after **entire program** is checked.
- **Debugging** is comparatively **hard**.
- Ex: C, C++.

Interpreter

- **Take single instruction as input**
- It is **Slower**
- **No** intermediate code is **generated**
- Required **less** memory As no intermediate code is generated
- **Every time** higher level program is converted into lower level program.
- **Errors** are displayed **for every instruction** interpreted.
- **Debugging** is **easy**.
- Ex: python, Ruby, basic.

Hybrid compiler

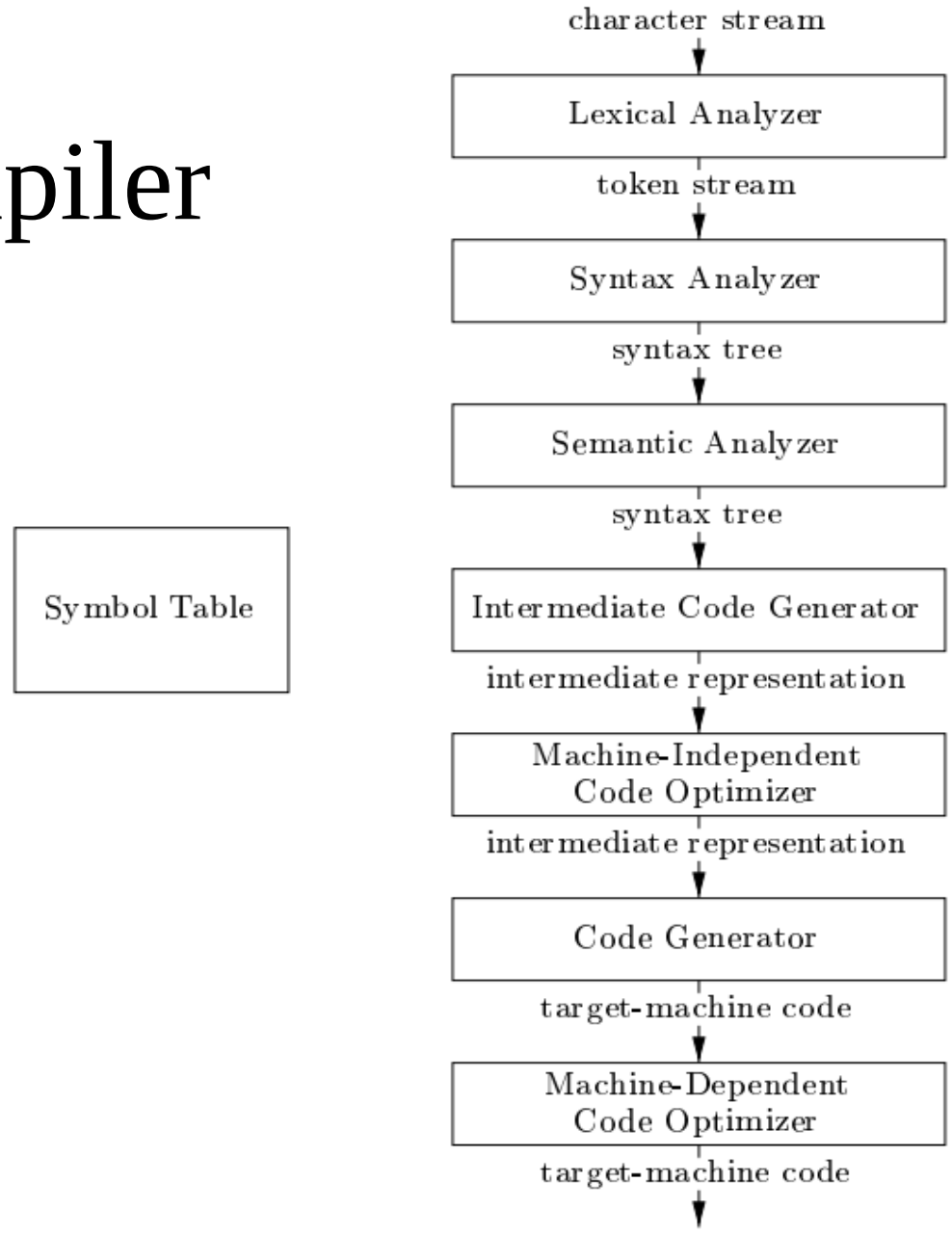
- Compilation and interpretation may be combined to implement a programming language:
  The compiler may produce intermediate-level code which is then interpreted rather than compiled to machine code.
  Ex: java



Java Language processor

- Java language processor has a special feature that combine both of the compiler and interpreter.
- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.
- In order to achieve faster processing of inputs to outputs, some Java compilers, called just-in-time compilers, translate the bytecodes into machine language immediately before they run the intermediate program to process the input.

Phases of a compiler



Lexical analysis (scanning)

- Break program down into its smallest meaningful symbols (tokens, atoms)
- Tokens include e.g.:
  - “Reserved words”: do - if - float - while
  - Special characters: ({ , + - = ! /
  - Names & numbers: myValue 3.07e02
- Start symbol table with new symbols found.
- lexical analyzer produces as output a token of the form

  <token- name, attribute-value>

Example (**position =initial + rate * 60**) (Lexical analyzer)

- “position” is a lexeme that would be mapped into a token <**id, 1** >, where **id** is an abstract symbol standing for **identifier** and **1** points to the symbol table entry for position.
- The symbol-table entry for an identifier holds information about the identifier, such as its name and type.

Example (**position =initial + rate * 60**)

- The assignment symbol “**=**” is a lexeme that is mapped into the token **<=>**. Since this token needs no attribute-value, we have omitted the second component. For notational convenience, the lexeme itself is used as the name of the abstract symbol.
- **“initial”** is a lexeme that is mapped into the token <**id, 2** > , where 2 points to the symbol-table entry for initial .
- “**+**” is a lexeme that is mapped into the token **<+>**.

Example (**position =initial + rate * 60**)

- **“rate”** is a lexeme that is mapped into the token **< id, 3 >** , where 3 points to the symbol-table entry for rate.
- **“*”** is a lexeme that is mapped into the token **< * >** .
- **“60”** is a lexeme that is mapped into the token **(60)**
- The produced token stream :

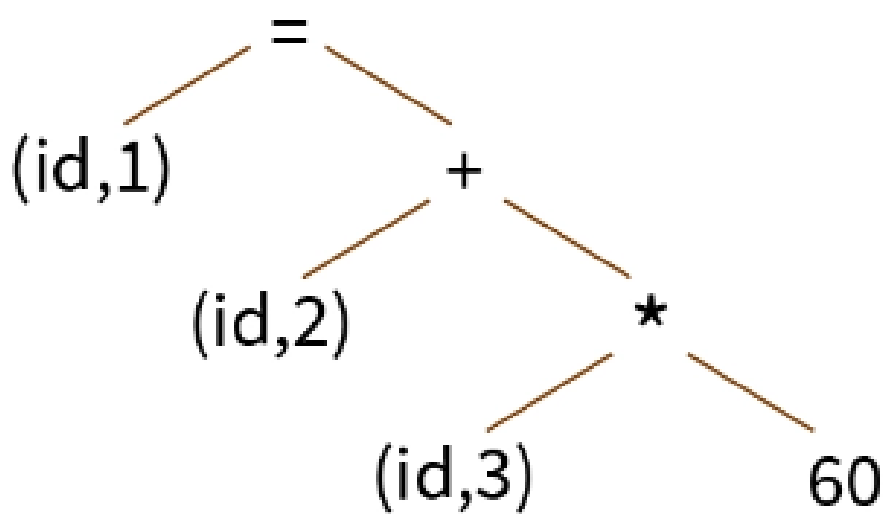
<id, 1> <= > <id, 2> <+> <id, 3> <*> <60>

Example (**position =initial + rate * 60**) (Syntax analyzer)

- The second phase of the compiler is syntax analysis or parsing. The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.
- A typical representation is a syntax tree in which each interior node represents an operation, and the children of the node represent the

  arguments of the operation.
- This tree shows the order in which the operations in the assignment **position = initial + rate * 60**

Generated syntax tree



Syntax Tree

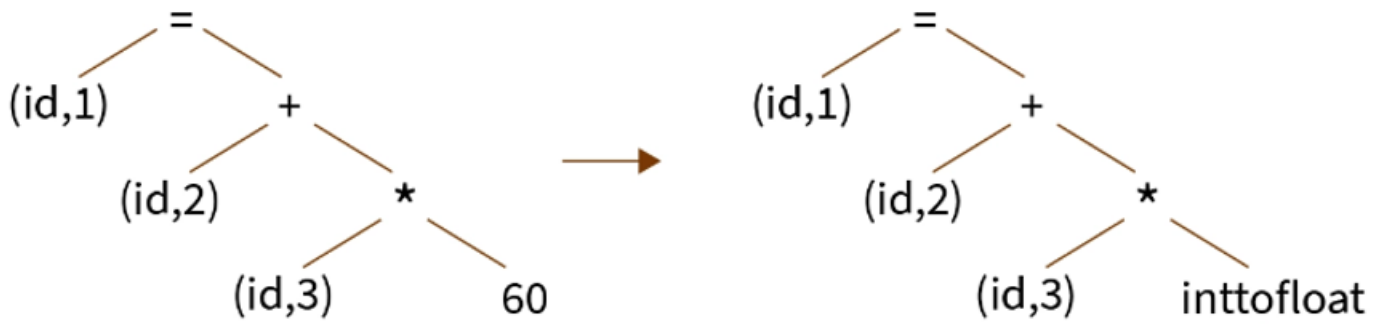
Semantic Analysis

- The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.
- It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.
- An important part of semantic analysis is type checking where the compiler checks that each operator has matching operands.
- For example, many programming language definitions require an array index to be an integer; the compiler must report an error if a floating-point number is used to index an array.

Type conversion

- The language specification may permit some type conversions called **coercions**.
- For example, a binary arithmetic operator may be applied to either a
- pair of integers or to a pair of floating-point numbers.
- If the operator is applied to a floating-point number and an integer, the compiler may convert or coerce the integer into a floating-point number.
- Suppose that position, initial, and rate have been declared to be floating-point numbers, and that the lexeme (60) by itself forms an integer.

Example (**position =initial + rate * 60**) (Semantic analyzer)



Syntax Tree **Semantic Tree**

Example (**position =initial + rate * 60**) Intermediate Code Generation

- After syntax and semantic analysis of the source program, many compilers generate an intermediate representation which called sometimes (three-address instruction) such as:

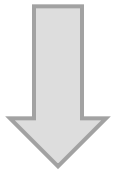
```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Code optimization

- The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result.
- Usually better mean faster, but other objectives may be desired, such as shorter code, or target code that consumes less power. The optimization of the intermediate code above can be as follows:

Example (**position =initial + rate * 60**) Code optimizer

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```



```
t1 = id3 * 60.0
id1 = id2 + t1
```

Three-address instruction forms

three-address instruction forms:

1. Assignment instructions of the form x = y op Z

   Ex: t2 = id3 * t1 , t3 = id2 + t2

2. Assignments of the form x = op y.

   Ex : t1 = inttofloat(60) or t6=-t5

3. Copy instructions of the form x = y

   Ex : id1 = t3

Three-address instruction forms cont.

4. An unconditional jump goto L. The three-address instruction with label L is the next to be executed.

L : t1 = t2 + id2

goto L

Three-address instruction forms cont.

- 5. Conditional jumps such as if x relop y goto L, which apply a relational operator < , <= , ==, >=, etc.) to x and y, and execute the instruction with label L next if x stands in relation relop to y . If not, the three-address
- instruction following if x relop y goto L is executed next, in sequence.

L : t1 = t2 + id2

if t3 < v goto L

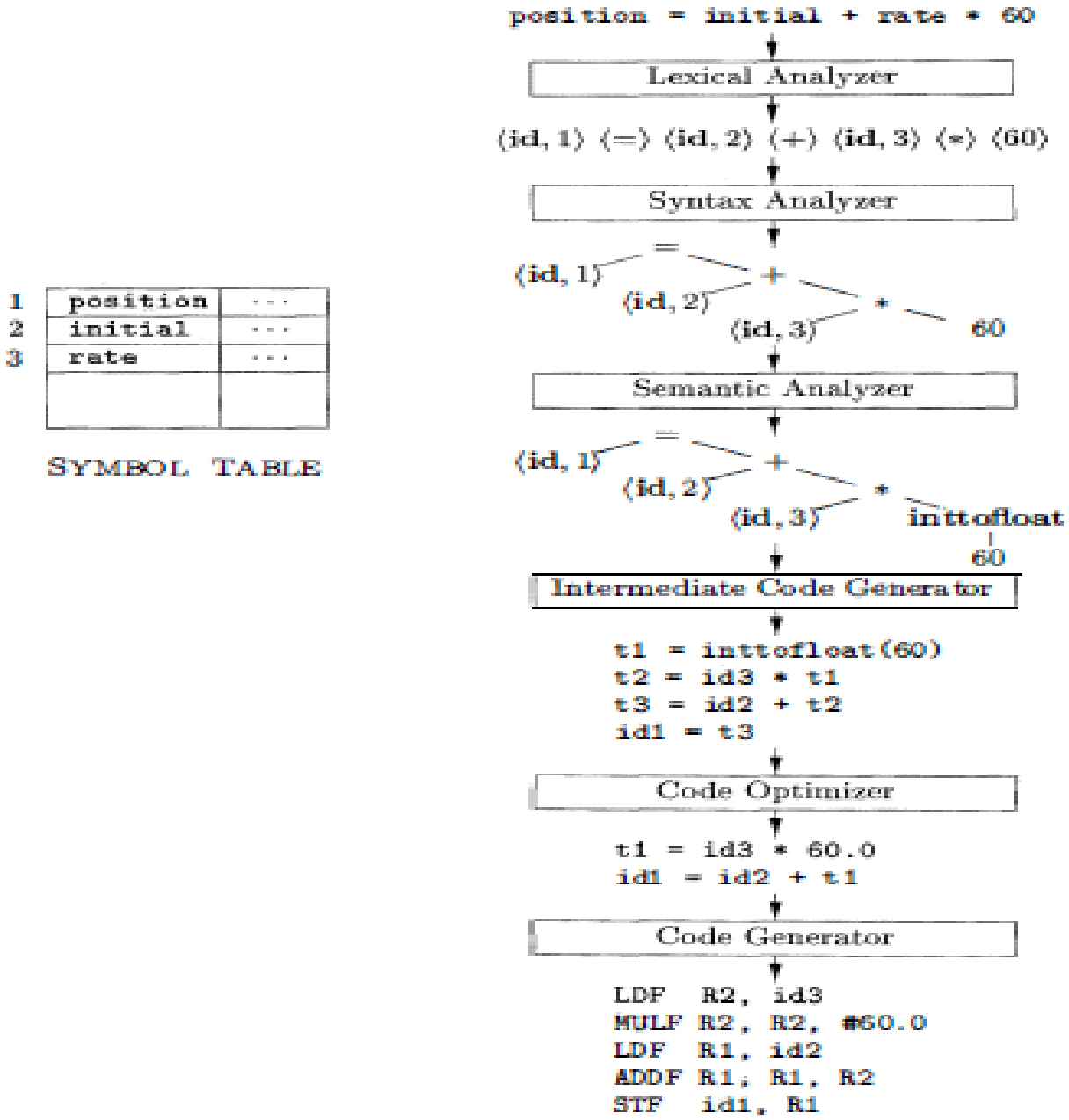
t1 = t2 + 1

Example (**position =initial + rate * 60**) Code generator

```
t1 = id3 * 60.0
id1 = id2 + t1
```

The output of the code generation step is as follows:

```
LDF   R2, id3
MULF  R2, R2, #60.0
LDF   R1, id2
ADDF  R1, R1, R2
STF   id1, R1
```



1	position	···
2	initial	···
3	rate	···

SYMBOL TABLE

Compiler-Construction Tools

1. **Parser generators** that automatically produce syntax analyzers from a grammatical description of a programming language.
2. **Scanner generators** that produce lexical analyzers from a regular-expression description of the tokens of a language.
3. **Syntax-directed translation engines** that produce collections of routines for walking a parse tree and generating intermediate code.
4. **Code-generator generators** that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.
5. **Data-flow analysis engines** that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.
6. **Compiler-construction toolkits** that provide an integrated set of routines for constructing various phases of a compiler.

Why learn about compilers?

- It is considered a topic that you should know in order to be “well-cultured” in computer science.
- A good craftsman should know his tools, and compilers are important tools for programmers and computer scientists.
- The techniques used for constructing a compiler are useful for other purposes as well.
- There is a good chance that a programmer or computer scientist will need to write a compiler or interpreter for a domain-specific language.

The Evolution of Programming Language

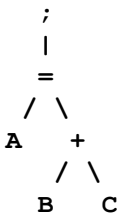
classification by generation

- **First-generation languages:** machine languages
- **Second-generation** : assembly languages
- **Third-generation** : higher-level languages like Fortran, Cobol, Lisp, C, C++, C#, and Java.
- **Fourth-generation languages: l**anguages designed for specific applications like NOMAD for report generation, SQL for database queries, and Postscript for text formatting.
- **fifth-generation language** has been applied to logic- and constraint-based languages like Prolog and OPS5.

Impacts on Compilers

- The advances in programming languages placed new demands on compiler writers.
- Compiler writers would take maximal advantage of the new hardware capabilities.
- Good software-engineering techniques are essential for creating and evolving modern language processors.

The Phases of a Compiler

Phase	Output	Sample
Programmer	Source string	`A=B+C;`
Scanner (performs lexical analysis)	Token string	`'A', '=', 'B', '+', 'C', ';'` And *symbol table* for identifiers
Parser (performs syntax analysis based on the grammar of the programming language)	Parse tree or abstract syntax tree	
Semantic analyzer (type checking, etc)	Parse tree or abstract syntax tree	

Intermediate code generator	Three-address code, quads, or RTL	`int2fp B t1`
		`+ t1 C t2`
		`:= t2 A`

Optimizer	Three-address code, quads, or RTL	`int2fp B t1` `+ t1 #2.3 A`

Code generator	Assembly code	`MOVF #2.3,r1`
		`ADDF2 r1,r2`
		`MOVF r2,A`

Peephole optimizer	Assembly code	`ADDF2 #2.3,r2` `MOVF r2,A`

The Grouping of Phases

- **Compiler** front and back ends:
  - Analysis (*machine independent* front end)
  - Synthesis (*machine dependent* back end)
- **Passes**
  - A collection of phases may be repeated only once (*single pass*) or multiple times (*multi pass*)
  - Single pass: usually requires everything to be defined before being used in source program
  - Multi pass: compiler may have to keep entire program representation in memory

Compiler-Construction Tools

Software development tools are available to implement one or more compiler phases:

- Scanner generators
- Parser generators
- Syntax-directed translation engines
- Automatic code generators
- Data-flow engines

Summary

- **Language Processors:** An integrated software development environment includes many different kinds of language processors such as compilers, interpreters, assemblers, linkers, loaders, debuggers, profilers.
- **Compiler Phases:** A compiler operates as a sequence of phases, each of which transforms the source program from one intermediate representation to another.
  - Lexical Analyzer
  - Syntax Analyzer
  - Semantic Analyzer
  - Intermediate Code Generator
  - Machine-Independent Code Optimizer
  - Code Generator
  - Machine-Dependent Code Optimizer
- **Machine and Assembly Languages:** Machine languages were the first generation programming languages, followed by assembly languages. Programming in these languages was time consuming and error prone.